/date

Microarchitecture

We have learned how to make digital building blocks & we have learned about RISC-V.

Now how to configure those Blocks in a way that it allows us to use RISC-V architecture?

Microarchitecture: Implementation of an architecture in hardware.

There can be multiple ways to implement it

- **Single cycle:** Each instruction executes in a single cycle
- **multicycle:** Each instruction is broken up into a series of shorter steps.
- **Pipelined:** Each instruction broken up into a series of steps & multiple instructions execute at once.

BHATTI

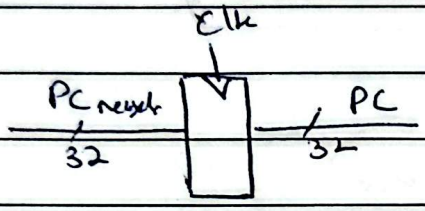
CamScanner

day/date

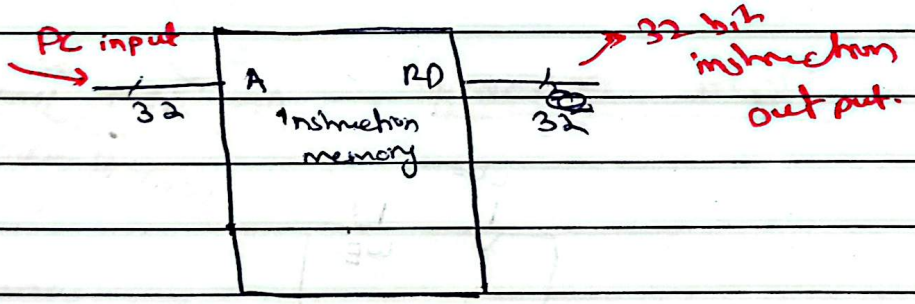
Components we'll use.



1) PC counter



2) Instruction memory.

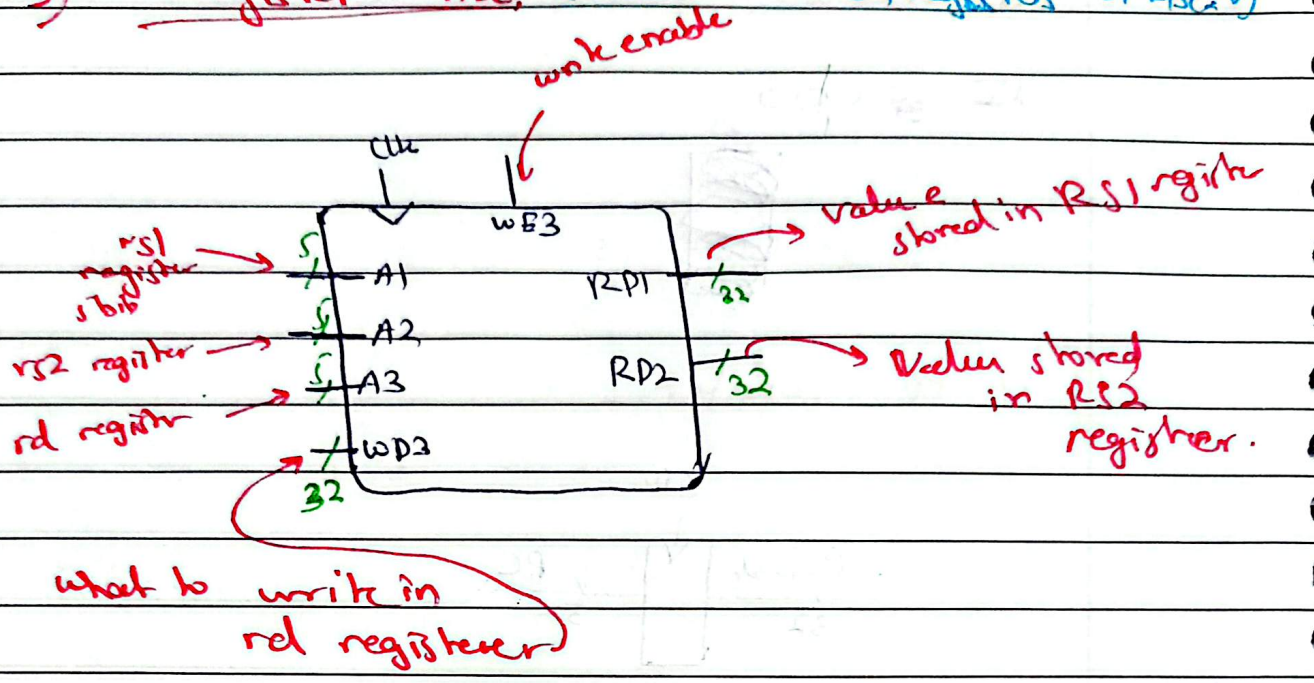


BHATTI

CamScanner

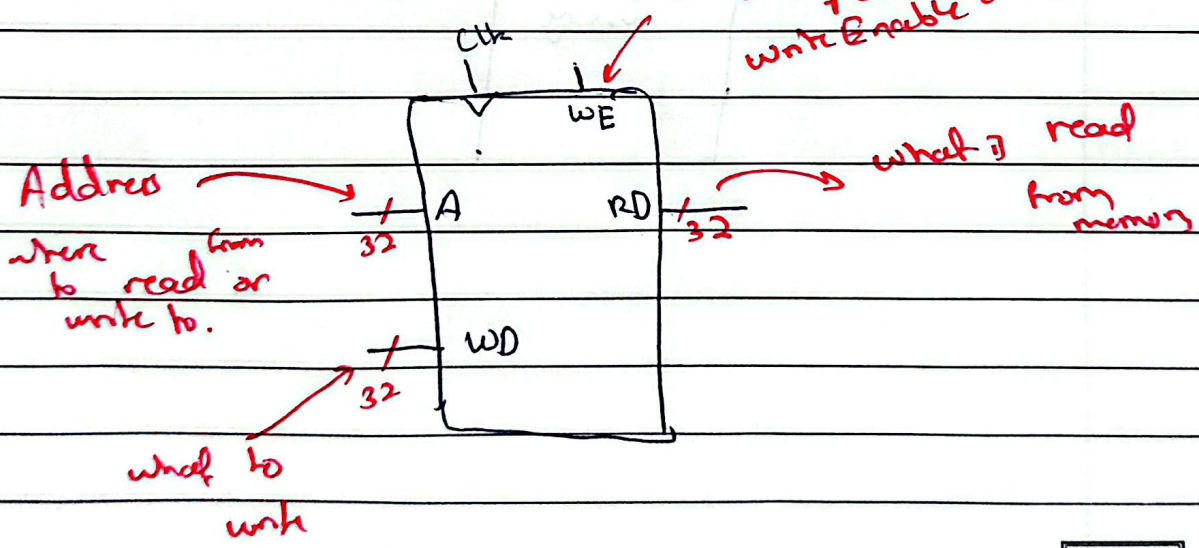
/date

3) Register file: (contains all 32 registers of RISC-V)



If we want to write to rd register WE3 must be on.

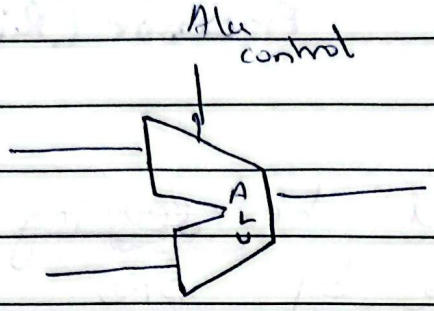
4) Data memory.



BHATTI

date

5) ALU.



Imp:

"A" in all these diagrams stand for address.

e.g in register file, the registers are denoted by a 5 bit address so they only need 5 bits but in memory, data is deposited/pointed by 32 bits so 32 bits needed.

BHATTI

CamScanner

late

Microarchitecture:

Single cycle Processor (Risc-V)

Now we will be ~~seeing~~ seeing instructions one by one & make an architecture that supports all.

But in an architecture there are 2 parts:

1) datapath
2) control path.

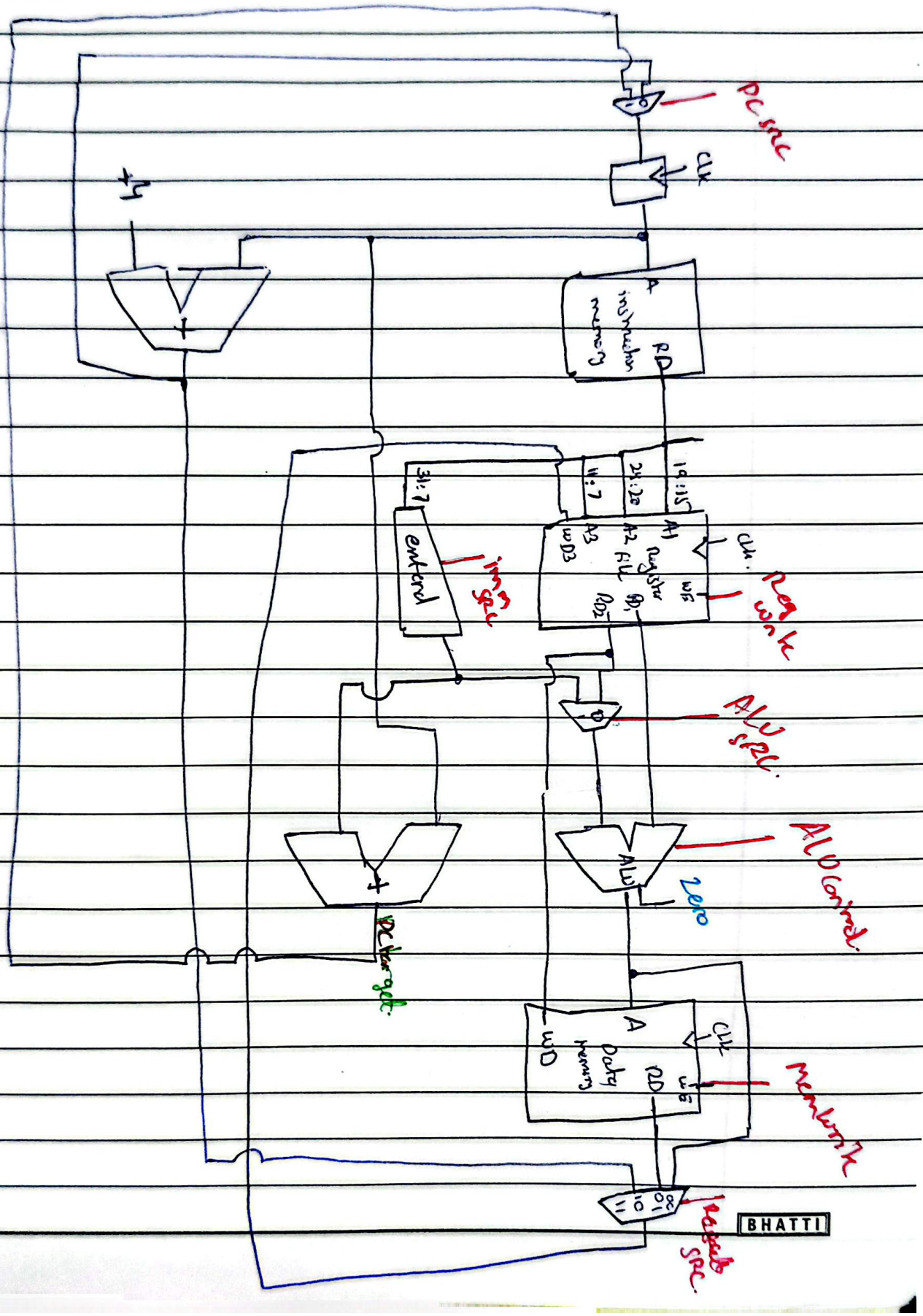
we will be working on the datapath first.

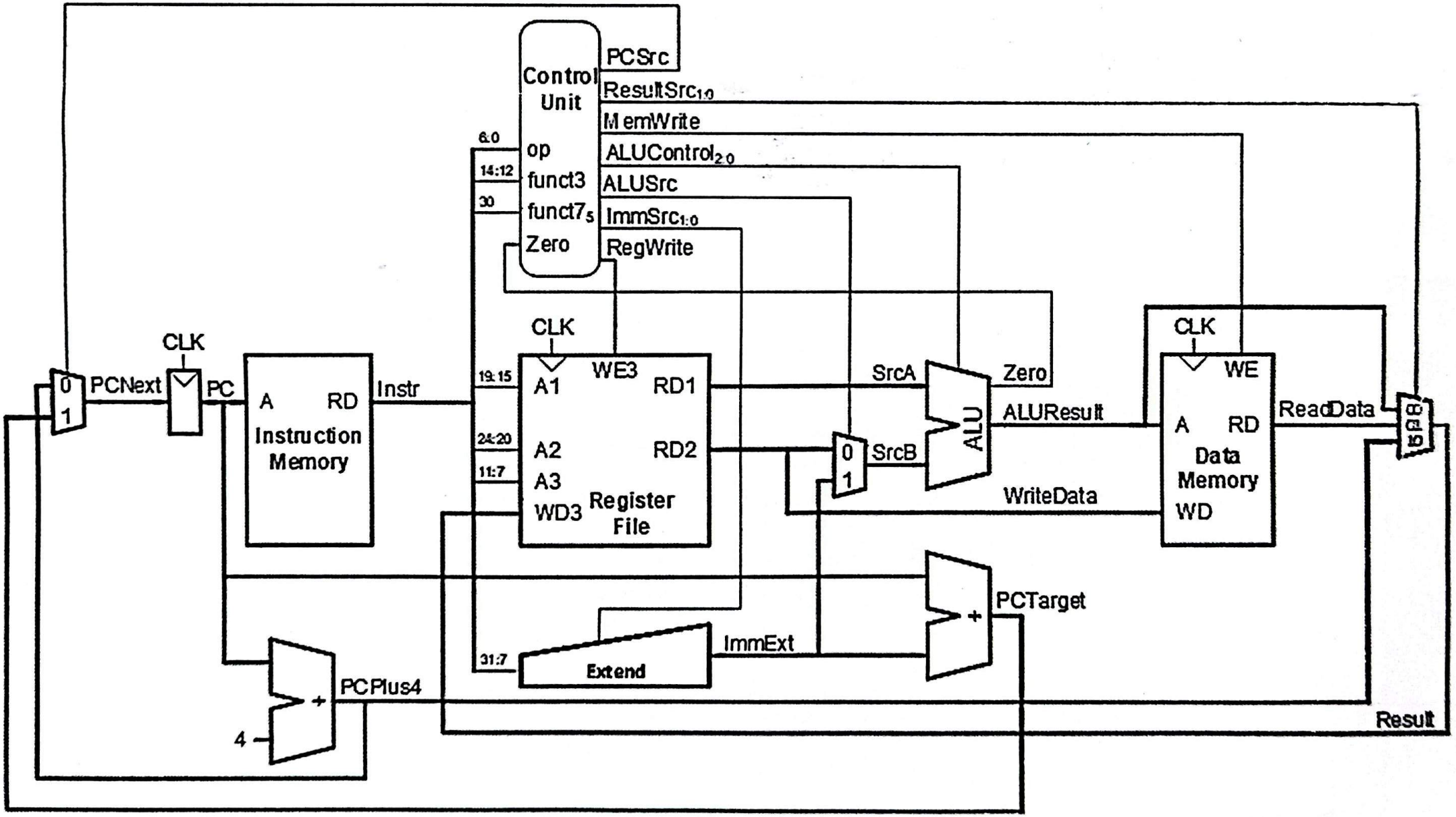
BHATTI

CamScanner

day/date

Full architecture:




Control Unit
PCSrc
ResultSrc1:0
MemWrite
ALUControl2:0
ALUSrc
ImmSrc1:0
RegWrite
Zero
op
funct3
funct7₅
6:0
14:12
30
CLK
PCNext
PC
A
RD
Instruction Memory
Instr
19:15
24:20
11:7
A1
A2
A3
WD3
WE3
RD1
RD2
Register File
SrcA
SrcB
ALU
ALUResult
WriteData
WE
Data Memory
WD
ReadData
Result
PCTarget
ImmExt
31:7
Extend
PCPlus4
4
0
1

y/date

Understanding:

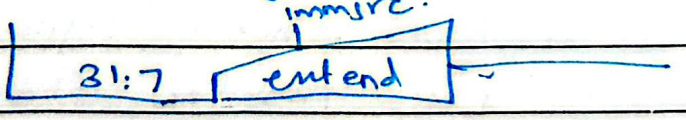
First of all let's understand how instruction actually comes:

- In figure we can easily see PC counter comes to instruction memory which then tells the instruction.

- Then that instructions bits are divided according to:
  1) opcode
  2) funct3
  3) funct7
  4) rs1
  5) rs2
  6) rd
  7) immediate

immediates:

- coming to immediate:
  This is handled by the block below Register file:



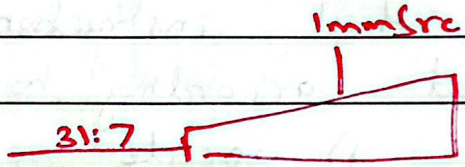
BHATTI

day/date

now this block gets all bits from 7 to 31 but every instruction type has different placements of immediate

e.g in I-type, imm is last 12 bits but in S-type it is imm 11:7 & imm 31:25

so for this we have the control bits in it "immSrc"



immSrc handles which bits are chosen as immediate.

imm Src 1:0	Instruction type
00	I type
01	S-type
10	B-type
11	J-type

* U-type is being ignored right now or else immSrc would become 3 bit.

BHATTI

- R type doesn't need immediate so is ignored & it get don't care (xx) condition on ImmSrc.

Register value:

Then comes register file, it has all the 32 registers init & can be addressed through 5 bits.

rs1 goes in A1
rs2 goes in A2

These values respectively go out of RD1 & RD2.

rs1 —A1 value→ RD1
rs2 —A2 value→ RD2

ALU:

Now all instruction almost use ALU eg in R type, rs1 + rs2 or rs1 - rs2 etc. in I type we need to do rs1 + immediate. or subtract or shift etc.

BHATTI

CamScanner

So for all these an ALU is needed inside.

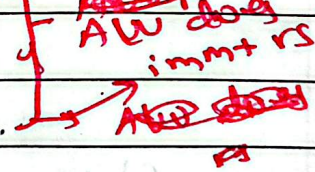
In all instructions an ALU operation happens only with rs1 and rs2 (rs2 or imm).

So to handle which to choose from rs2 and immediate we use a mux controlled by ALUSrc.

What happens with ALU result?

consider instructions:

1) lw x2, x4(0)
2) sw s0, s1(4)
3) addi t0, s0, 4.



in 1) we need to read the memory pointed by ALU result.

2) we need to write to memory where ALU result point.

3) we need to write in output

BHATTI

CamScanner

e

okay so now we need to handle 3 things:

1) write to memory
2) Read from memory
3) write to register.

Data Memory:

To control Read or write we have write Enable (WE) bit.

now we only write to memory in one instruction type → S-type & in that rs2/RD2 is written to memory as

sw rs2, (rs1)offset.

so simply we connect RD2 to WD of write memory.

As for read from data, if write enable bit is off, it see the ALU result as address, from where to read & sends that data out of RD.

BHATTI

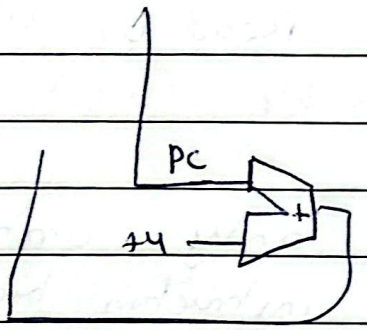
CamScanner

day/date

How is PC+4 handled?

After every instruction PC+4 needs to happen to control next instruction.

That is controlled by an adder just below the instruction memory.



How are PC handled when branching or jmps(jal) happen?

We know both beq & jal use PC-addressing, which is PC + offset.

so now this needs to be handled as well.

BHATTI

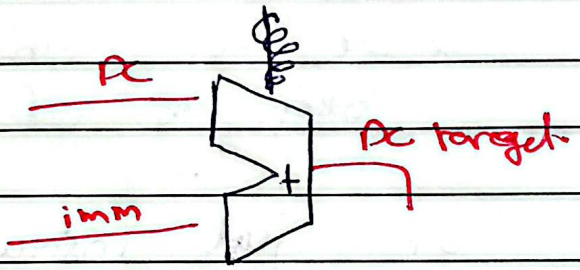
CamScanner

late

offset is the immediate value of the instruction beq or jal.

& we already have PC value.

So if you see in diagram it is handled by the Adder below the ALU:



But how to decide which PC value to take? PC target or PC+4?
This is handled by mux behind the PC counter.

BHATTI

CamScanner

lay/date

That mux is controlled by a PCSRC.

So

If PCSRC = 1, use PCtarget (beq, jal)
if PCSRC = 0, use PC + 4.

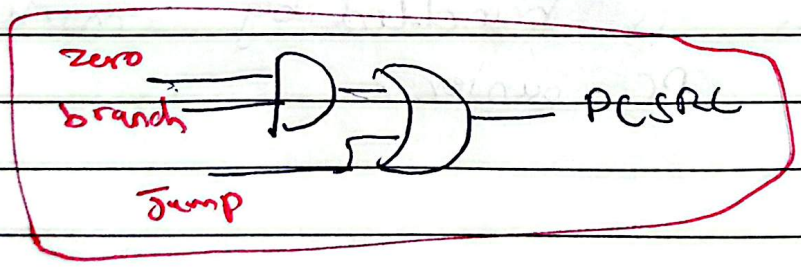
How is PCSRC decided?

now PCSRC has 2 ways:

either branch instruction (beq) with zero flag = 1.
* we are only considering beq ops of yet.

or jal instruction comes.

so this zero flag comes from the ALU output of rs1 - rs2.



BHATTI

day/date

In jal, the command is jal ra, label. So how is this ra getting PC+4 value?

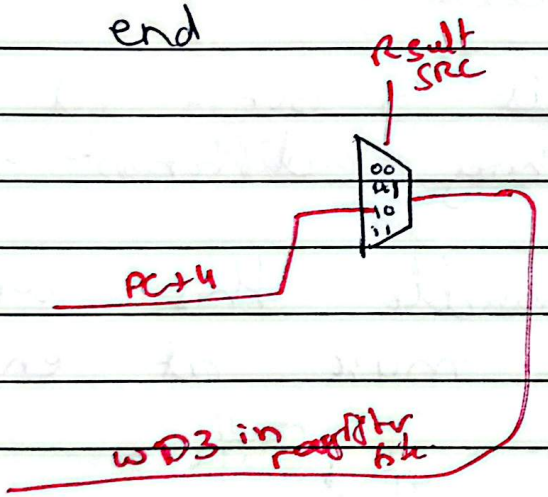
in jal ra, label

or

jalr ~~rd~~, rs1, immediate/label.

our rd need to get PC+4 value. stored.

This is handled by our mux at end



remember jalr is not handled by the current architecture

our this last end mux whatever output it gets writes to the rd register & so in its 3 input we placed PC+4 value. & in jalr or jal it can be triggered by Result SRC.

BHATTI

CamScanner

y/date

Writing to a Register:

Consider :

ALU output

1) add s0, s1, t0

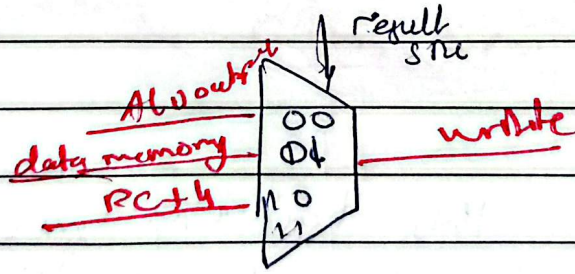
lw s0, (t0)4 — memory → data

jal ra, 0x45321

ra ← PC+4

In all 3 our rd gets something different.

To handle this we have a mux at end.



BHATTI

date

[redacted]

[redacted]:

Is Jalr instruction handled?

No it is not.

jalr rd, (rs1) offset.

now in it PC = (rs1) offset
but we haven't added this
connection in the architecture
yet so it won't work.

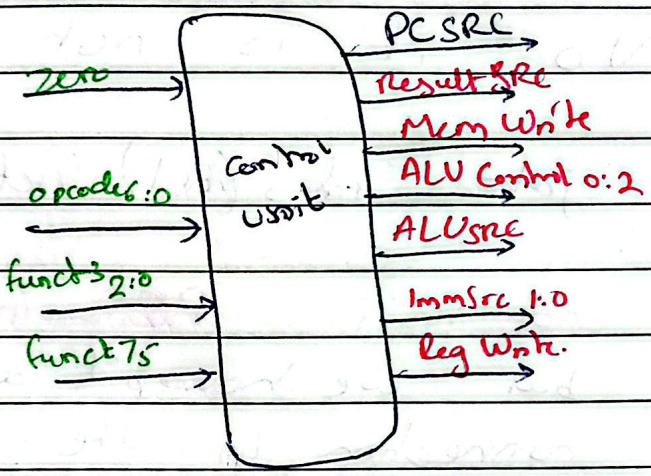
BHATTI

CamScanner

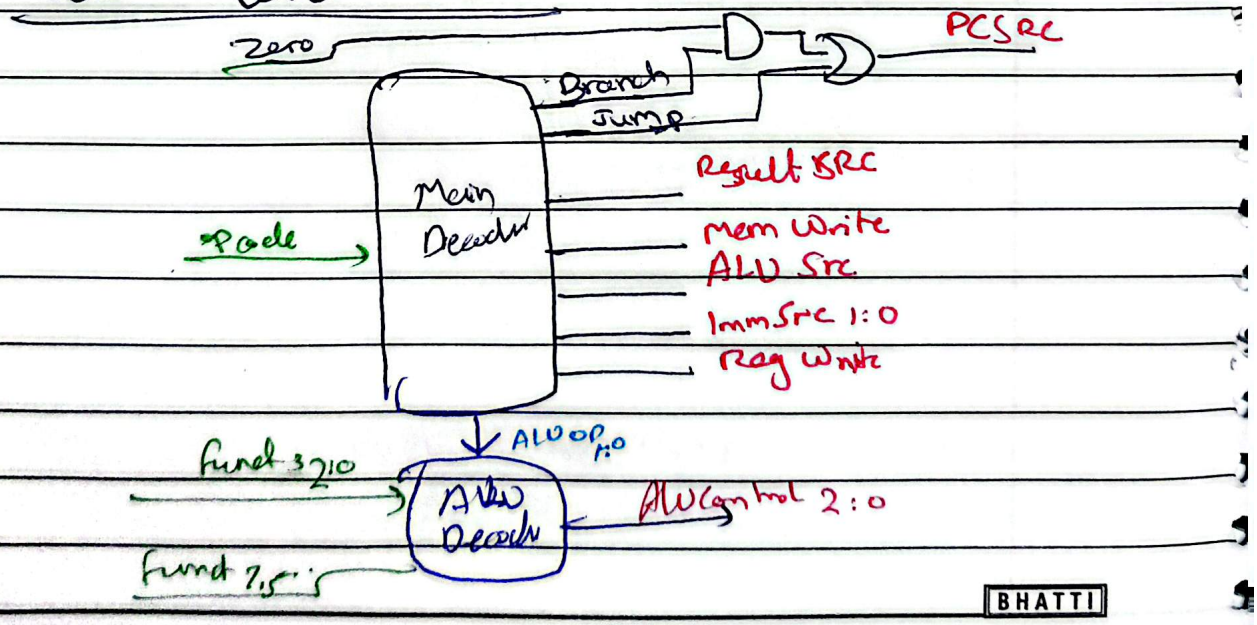
lay/date

Control Unit:

All these instruction and which to run and how to control the multiplexers are decided by CU.



low level view:



BHATTI

e

Inputs

1) opcode 0:6:

These are the first 7 bits of the machine code

2) funct3 2:0:

Helps in decoding the perticular ALU operation.

3) funct7 5:

only 5th bit of the funct7 is used to help in ALU.

BHATTI

CamScanner

e

Outputs.

1) PC Src:

For PC mux.

Tells which PC to take next, either PC + 4 or PC + Offset (beq, jmp).

2) Result Src:

Tells what to write (from mux) in the rd register if need to be written.

3) Mem Write

Memory write enable. If one write to memory, if 0 don't write, read only.

4) ALU Src:

Tells mux before ALU to decide imm or rs2.

5) Imm Src 1:0:

Tells out of the (31:7) bits what bits to pick for immediate.

BHATTI

CamScanner

te

6) Reg Write.

Tell if we are allowed to write in a register or not

7) ALU control:

Tell ALU which specific operation to perform e.g Add, Sub, or, and, slt etc.

ALU decoder & ALU OP:

The control units has 2 parts:

- main decoder
- ALU decoder.

The ALU decoder gets a 2 bit input (ALUOP) from maindecoder

BHATTI

ay/date

The ALU that we are currently working has 5 operations: (and, or, sub, slt, add).

Now ALU OP tells us what type of instructions is there.

	ALU OP
lw	00
sw	00
R-type	10
I-type	10
beq	01
Jal	XX (don't care).

How does ALUOP help?

When ALU OP sends "00" code this means lw or sw so Always use Add in this.

If ALU OP sends "01", this means beq. so Always Subtract in it.

BHATTI

CamScanner

te

If ALU OP sends "10", now use the funct 3 & funct 7 to decide which operation (add, sub, slt, or, and) to perform.

ALUOp	funct3	op_5, $funct7_5$	Instruction	$ALUControl_{2:0}$
00	x	x	lw, sw	000 (add)
01	x	x	beq	001 (subtract)
10	000	00, 01, 10	add	000 (add)
	000	11	sub	001 (subtract)
	010	x	slt	101 (set less than)
	110	x	or	011 (or)
	111	x	and	010 (and)

CamScanner

/date

Complete control table for all functionalities (by current architecture):

op	Instruction	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
3	lw	1	00	1	0	01	0	00	0
35	sw	0	01	1	1	xx	0	00	0
51	R-type	1	xx	0	0	00	0	10	0
99	beq	0	10	0	0	xx	1	01	0
19	I-type	1	00	1	0	00	0	10	0
111	jal	1	11	x	0	10	0	xx	1

* jal doesn't use ALU so it becomes don't care.

BHATTI

CamScanner